StackMaster Tech

CLOUD · DATA · OBSERVABILITY · SRE

FIELD HANDBOOK · DAY 4 · SPLUNK POWER USER TRACK

Reporting, Analysis & Dashboards

Turning results into insight: the **chart** and **timechart** reporting commands, calculating and formatting with **eval**, and building interactive **Dashboard Studio** apps with inputs, drilldowns and dynamic visualizations.

chart · timechart | Split Series · Formatting | eval Calc & Format

Dashboard Studio | Inputs · Drilldowns | Dynamic Coloring

Vivek Arora
Corporate Trainer, Author & Consultant
StackMaster Tech · Bengaluru, India

youtube.com/@StackMasterTech
github.com/vivek9325
linkedin.com/in/vivek11arora

Contents

01 · From Results to Insight

A table of numbers answers a question; a well-built dashboard lets someone *explore* the answer. Day 4 completes the journey: shape results with the **reporting commands** (`chart`, `timechart`), refine and format them with **eval**, then assemble them into interactive **Dashboard Studio** apps whose inputs, drilldowns and dynamic visualizations turn a static report into a tool.

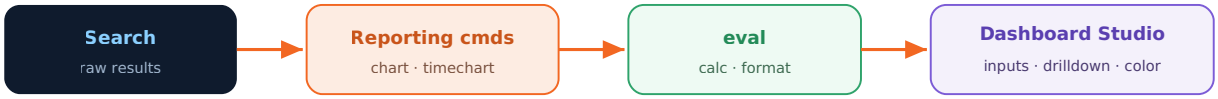


Fig 1.1 — Day 4's pipeline: from a search to an interactive app.

The dataset, still Buttercup Games

Every chart, eval and dashboard in this book runs on the same data from Days 1–3: `index=web` (`access_combined` purchases and `checkout_json` baskets) and `index=security`. Revenue, categories, actions and HTTP status codes drive the visuals.

CLASSIC DASHBOARDS VS DASHBOARD STUDIO
Splunk has two dashboard frameworks: the older **SIMPLE XML** (Classic) and the newer **DASHBOARD STUDIO** (JSON, grid or absolute layout, richer visuals). This day uses **DASHBOARD STUDIO**; the concepts — inputs, tokens, drilldowns — carry over to Classic, though the syntax differs.

PART A

Reporting Commands

Reporting commands transform events into the tabular shape a visualization needs. **chart** aggregates across one or two dimensions; **timechart** does the same with time always on the x-axis. Master how they split series, drop noise, and format — and you can draw almost anything.

02 · Exploring Available Visualizations

A visualization is only as good as the fit between the *shape of your results* and the *chart type*. Splunk offers a gallery of built-ins; the trick is knowing which expects which table shape.

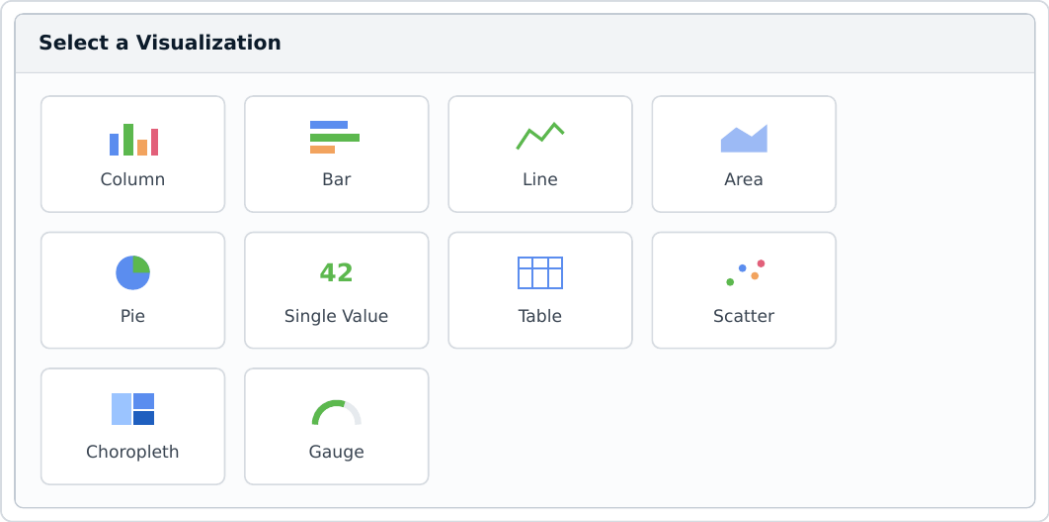


Fig 2.1 — The visualization picker. Each type expects a particular result shape.

Visualization	Expects	Good for
Column / Bar	category + one or more measures	Comparing values across categories
Line / Area	`_time` + measures	Trends over time
Pie	one category + one measure	Part-to-whole (few slices)
Single Value	one number (+ trend)	KPIs, headline metrics
Table	any rows/columns	Detail, mixed types, drilldown source
Choropleth / Cluster map	region or lat/long	Geographic distribution
Scatter / Bubble	two-three measures	Correlation, outliers

SHAPE FIRST, CHART SECOND

Decide the story ("revenue by category over time"), which fixes the result shape (`_time × categoryId → sum(price)`), which fixes the chart (multi-series line). Work in that order and you never fight the visualization.

03 · Creating Charts & Splitting into Series

`chart` aggregates a measure across categories. With one `BY` clause you get a single series; the magic is the **split** — `chart <agg> OVER <x> BY <split>` — which turns one bar per category into a grouped or stacked set, one colour per split value.

A basic chart

```
index=web | chart count BY categoryId
```

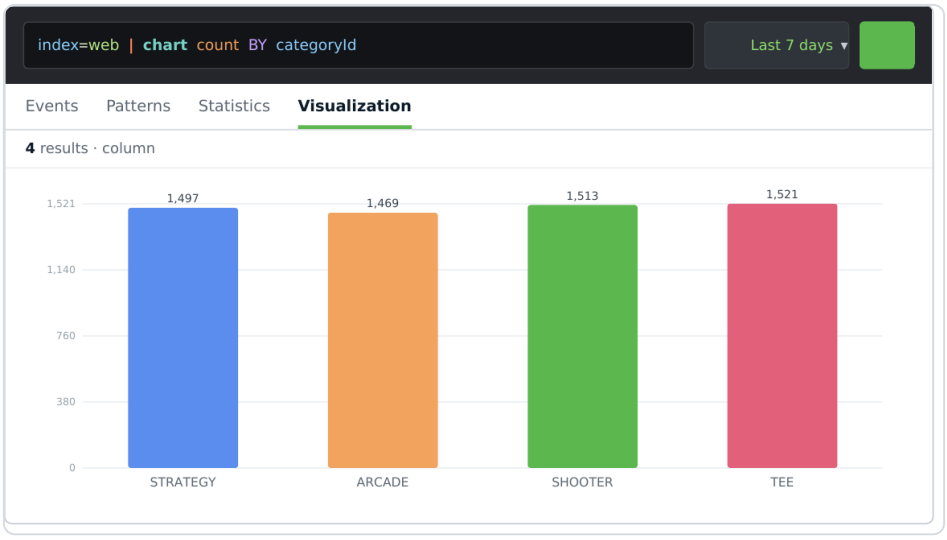


Fig 3.1 — One measure across one dimension: events per category.

Split into multiple series

The `OVER … BY …` form places the first field on the x-axis and creates one series per value of the second:

```
index=web | chart count OVER categoryId BY action
```

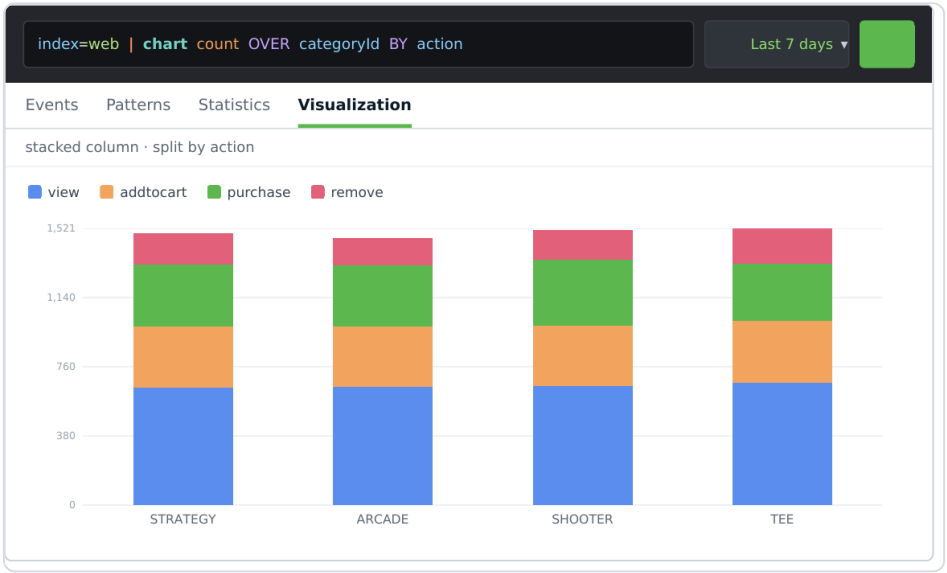


Fig 3.2 — Split by action: each category stacks view / addtocart / purchase / remove.

CHART VS STATS

`stats … BY a, b` yields a long, tidy table (one row per a-b pair); `chart … OVER a BY b` yields a wide, pivoted table (one row per a, one column per b) — exactly what grouped/stacked visualizations want.

CARDINALITY GUARD

A split field with hundreds of values makes an unreadable chart and a huge table. Splunk caps split columns (default ~10) and buckets the rest as `OTHER` — which the next chapter shows how to control.

04 · Omitting Null & Other Values

Two kinds of noise clutter split charts: a **NULL** series (events missing the split field) and the **OTHER** bucket (everything beyond the column limit). `chart` and `timechart` expose options to drop or reshape both.

Option	Default	Effect
`usenull`	true	Set `usenull=f` to drop the NULL series entirely.
`useother`	true	Set `useother=f` to drop the OTHER bucket instead of lumping small values.
`limit`	10	`limit=5` keeps top 5 series; `limit=0` keeps all (no OTHER).
`nullstr / otherstr`	NULL / OTHER	Rename the buckets for a cleaner legend.

Example — clean top-5 series, no null, no other

```
index=web
| chart count OVER categoryId BY productId limit=5 useother=f usenull=f
```

This keeps only the five highest-volume products per category and suppresses both the catch-all OTHER column and any NULL series — a far more legible chart.

WHERE AFTER CHART

To trim by a threshold rather than a rank, follow the chart with `| where <series> > 100`, or use `fillnull value=0` to turn gaps into explicit zeros when a continuous line looks better than a broken one.

05 · timechart — Time-Series Charts

`timechart` is `chart` with time hard-wired to the x-axis. It buckets events by `span` and aggregates each bucket, producing the tidy time series that line and area charts want. A single `BY` field splits it into multiple lines on one timeline.

Single series

```
index=web action=purchase | timechart span=1d sum(price) AS revenue
```

Multiple values on the same timeline

Give `timechart` several aggregations, or a `BY` split, to plot multiple lines together:

```
# split one measure into a line per category
index=web action=purchase | timechart span=1d sum(price) BY categoryId

# or several measures on one timeline
index=web | timechart span=1h count AS hits, dc(clientip) AS visitors
```

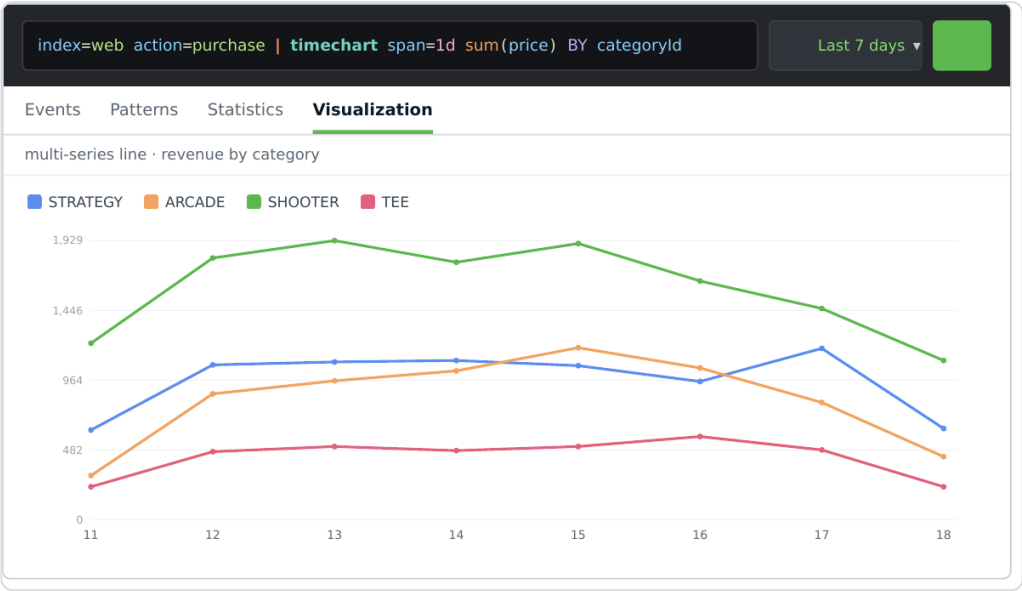


Fig 5.1 — Revenue per day split by category — four lines, one timeline.

LET SPAN BREATHE

Omit `span` and Splunk auto-selects a sensible bucket for the time range. Set it explicitly (`span=1d`, `span=15m`) when you need consistent buckets across dashboards or comparisons.

TWO AXES

When series have very different scales (revenue vs visitor count), use the visualization's **OVERLAY / SECOND Y-AXIS** option so the small series isn't flattened against the large one.

06 · Formatting Charts

Formatting is where a chart becomes readable. Most of it lives in the **Format** menu (Classic) or the visualization's **configuration panel** (Studio), but many choices are driven by the search itself — field names become axis and legend labels, so name them well.

Control	What it changes
Chart type & stacking	Column/bar/line/area; stacked vs grouped vs 100%-stacked.
Axis titles & scale	Titles, min/max, linear vs log, unit labels.
Legend	Position, truncation, show/hide.
Number format	Thousands separators, decimals, units — often set via `eval` + `tostring` (Ch. 9).
Colours	Series palette; static or dynamic (Part E).
Null handling	Gaps, zero, or connected lines.

Formatting starts in the SPL

```
# rename fields so the axis/legend read cleanly
index=web action=purchase
| timechart span=1d sum(price) AS "Daily Revenue (USD)" BY categoryId
```

CONSISTENCY ACROSS A DASHBOARD

Define units and number formats once (calculated fields, consistent `AS` names, a shared colour palette) so every panel speaks the same visual language — the mark of a professional dashboard.

07 · When to Use Each Reporting Command

Four commands cover most reporting. Choosing the right one is mostly about your x-axis and how many dimensions you're crossing.

Command	Produces	Use when
`stats`	Tidy table, one row per group	Numbers for further processing, or a detail table; any number of `BY` fields.
`chart`	Pivoted table (x = one field, columns = split)	Comparing a measure across a category, optionally split by a second — for bar/column.
`timechart`	Pivoted table with `_time` on x	Anything over time; trends, one split field for multiple lines.
`top` / `rare`	Ranked list with count & percent	"Top N" / "least common" — quick leaderboards.

The same question, three ways

```
# detail table
| stats sum(price) BY categoryId, action
# pivoted for a stacked column
| chart sum(price) OVER categoryId BY action
# leaderboard
| top categoryId limit=3
```

RULE OF THUMB

Time on the x-axis → `timechart`. A category on the x-axis → `chart`. Just the numbers → `stats`. A ranking → `top` / `rare`.

PART B

Analyzing, Calculating & Formatting with eval

Day 1 met **eval** as a function library; here it's a reporting tool. Compute new measures, convert units, round for readability, format for humans, branch with conditionals, and filter on the results of your own calculations — the connective tissue between a raw search and a polished panel.

08 · Calculations & Conversions

`eval` creates or overwrites a field from an expression evaluated per event. Arithmetic, unit conversion, and field-to-field math all live here — the raw material every measure is built from.

Arithmetic & unit conversion

```
index=web action=purchase
| eval tax       = price * 0.18,              # calculate
       total     = price * 1.18,              # calculate
       size_mb   = bytes / 1024 / 1024,       # convert bytes -> MB
       price_num = tonumber(price)              # cast string -> number
```

Need	Function / operator
Arithmetic	`+ - * / %`
Cast to number / string	`tonumber()` · `tostring()`
Absolute / power / sqrt	`abs()` · `pow()` · `sqrt()`
Time conversion	`strftime()` · `strptime()`

TYPES MATTER

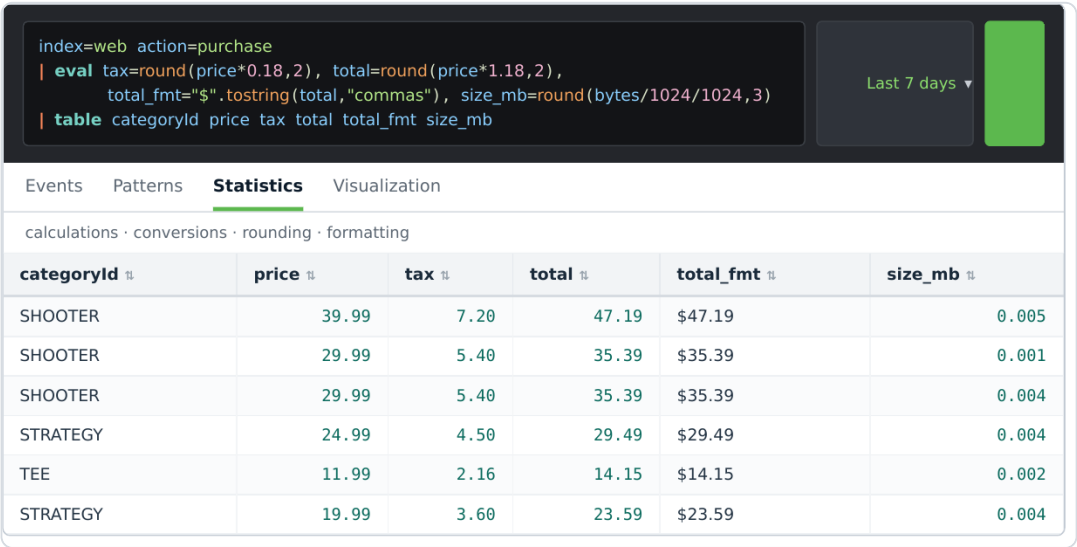
Fields extracted from text are strings. Arithmetic auto-coerces where it can, but comparisons and sorting don't — cast with `tonumber()` when a "9" must not sort after "10".

09 · Rounding & Formatting Values

Numbers that are correct can still be unreadable. `round()` controls precision; `tostring()` adds thousands separators, durations, or hex; string concatenation adds units and symbols.

Round, then format for humans

```
index=web action=purchase
| eval tax       = round(price*0.18, 2),
       total     = round(price*1.18, 2),
       total_fmt = "$" . tostring(total, "commas"),
       size_mb   = round(bytes/1024/1024, 3)
| table categoryId price tax total total_fmt size_mb
```



categoryId	price	tax	total	total_fmt	size_mb
SHOOTER	39.99	7.20	47.19	$47.19	0.005
SHOOTER	29.99	5.40	35.39	$35.39	0.001
SHOOTER	29.99	5.40	35.39	$35.39	0.004
STRATEGY	24.99	4.50	29.49	$29.49	0.004
TEE	11.99	2.16	14.15	$14.15	0.002
STRATEGY	19.99	3.60	23.59	$23.59	0.004

Fig 9.1 — Calculate, convert, round and format in one eval; total_fmt is display-ready.

tostring(x, …)	Result
`"commas"`	`1234567` → `1,234,567`
`"duration"`	seconds → `HH:MM:SS`
`"hex"`	integer → hexadecimal

ROUND FOR DISPLAY, KEEP PRECISION FOR MATH

Round in a *separate* display field and aggregate on the un-rounded value, so a column of rounded numbers still sums to the correct total.

10 · Conditionals & Filtering Calculated Results

Classification and filtering complete the toolkit. `if()` and `case()` branch a field on conditions; `where` then filters on *calculated* fields — something base-search keywords can't do, because those fields don't exist until eval runs.

Classify, then filter on the new field

```
index=web sourcetype=checkout_json | spath
| eval tier = if(cart.total >= 80, "large", "standard")
| where tier = "large"
| sort - cart.total
```

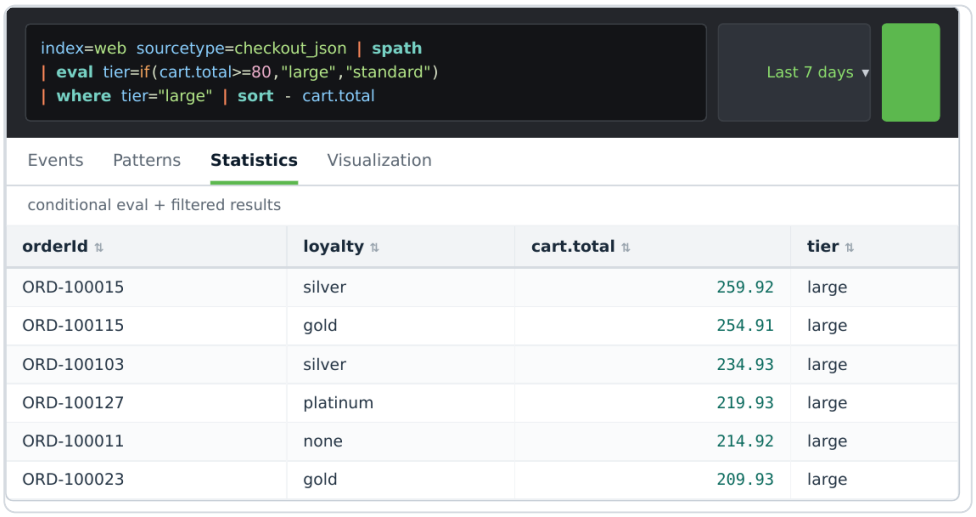


Fig 10.1 — if() assigns a tier; where keeps only large baskets — a filter on a computed field.

Multi-way with case()

```
| eval band = case(cart.total<40,"small", cart.total<80,"medium", true(),"large")
```

SEARCH VS WHERE

The base search and `| search` filter on *indexed/extracted* fields fast, early. `| where` evaluates an expression per row and is the only way to filter on eval-created fields or compare two fields (`where price > cat_avg`). Put cheap keyword filters first, `where` after the eval.

EVAL INSIDE STATS

Aggregate a condition directly with `count(eval(status>=400))` — no separate eval step needed for a one-off classification you only want to count.

PART C

Dashboard Studio — Inputs

A dashboard becomes a *tool* when the viewer can steer it. Inputs — dropdowns, time pickers, text boxes — set **tokens** that flow into panel searches. Here we build dynamic inputs populated from data, and chain them so one input filters the next.

11 · Dashboard Studio & Inputs

Dashboard Studio describes a dashboard as **JSON**: a set of **data sources** (searches), **visualizations** bound to them, a **layout** (grid or absolute), and **inputs**. An input captures a choice and stores it in a **token**; any search that references `$token$` re-runs when the token changes.

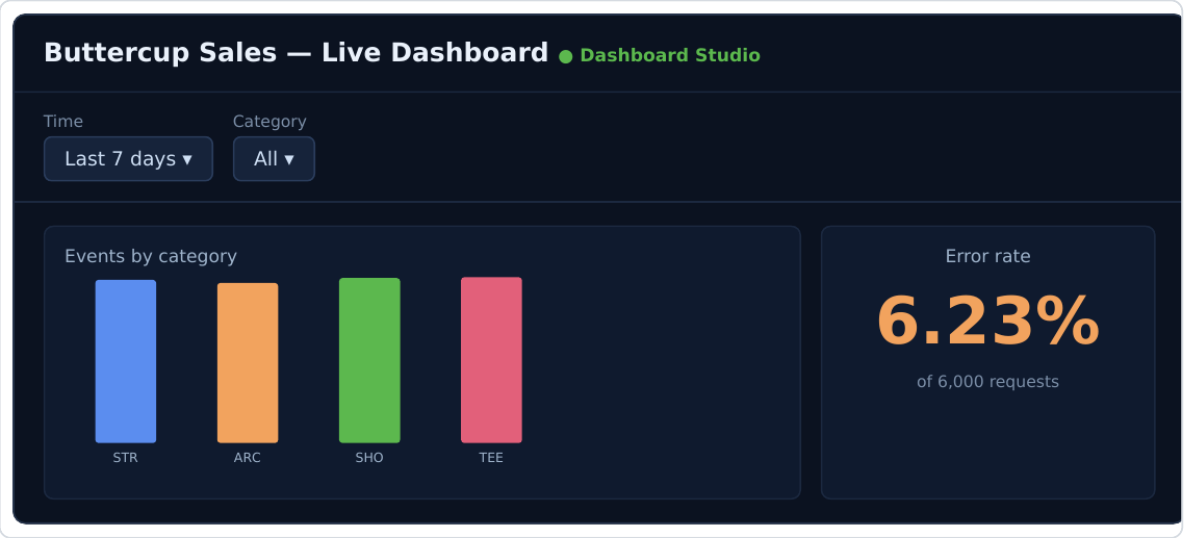


Fig 11.1 — A Studio dashboard: a time picker and a Category dropdown drive the panels below.

Types of input

Input	Token holds	Typical use
Time	earliest / latest	Global time range for the page
Dropdown	single value	Pick one category / host / status
Multiselect / Checkbox	multiple values	Choose several categories
Radio	single value	A small fixed set (span 1h/1d)
Text	free text	Search a substring, an IP
Link list	single value	Tabs / segmented control

How an input reaches a search

```
# the panel's data source uses the token set by the Category dropdown
index=web categoryId=$category$
| timechart span=1d count
```

DEFAULT & "ALL"
Give inputs a default token value so the dashboard renders before anyone touches it. For an "All categories" option, set the token to a wildcard (`*`) or omit the clause via a token that expands to nothing — Studio's *default* and *initial value* settings handle both.

12 · Dynamic & Cascading Inputs

A **dynamic input** populates its choices from a search instead of a hard-coded list, so it always reflects the data. A **cascading input** takes it further: its populating search references the token of an *earlier* input, so choosing a category narrows the product list to that category.

Dynamic — options from a search

```
# dropdown "Category" populated dynamically, de-duplicated and sorted
index=web | stats count BY categoryId | sort categoryId
# field for label -> categoryId, field for value -> categoryId
```

Cascading — the second input depends on the first

```
# dropdown "Product" populated using the Category token $cat$
index=web categoryId=$cat$ | stats count BY product_name | sort product_name
```

Fig 12.1 — Choosing SHOOTER repopulates Product with only that category's titles; both tokens feed the panel.

ORDER THE CHAIN

A cascading input's populating search runs whenever its upstream token changes. Set a default on the parent so the child has something to load with, and consider a "loading…" state for slow populating searches.

TOKEN HYGIENE

If a child token keeps a stale value after the parent changes, reset it in the parent's *change* handler (set child token to its default) so you never query `categoryId=SHOOTER product_name="Mediocre Kingdoms"` — a combination that returns nothing.

PART D

Dashboard Studio — Drilldowns

A drilldown is what happens when someone clicks. Done well, it turns a static panel into a launchpad — set a token and filter another panel, jump to a detail search, open another dashboard, or link out to a system of record. Tokens are the mechanism; context is the craft.

13 · Drilldowns & Tokens

When a user clicks a chart element or table row, Dashboard Studio exposes the clicked value through **click tokens** (e.g. `click.value`, `click.name`, `row.<field>`). A drilldown reads those, does something, and — for in-page drilldowns — sets tokens that other panels consume.

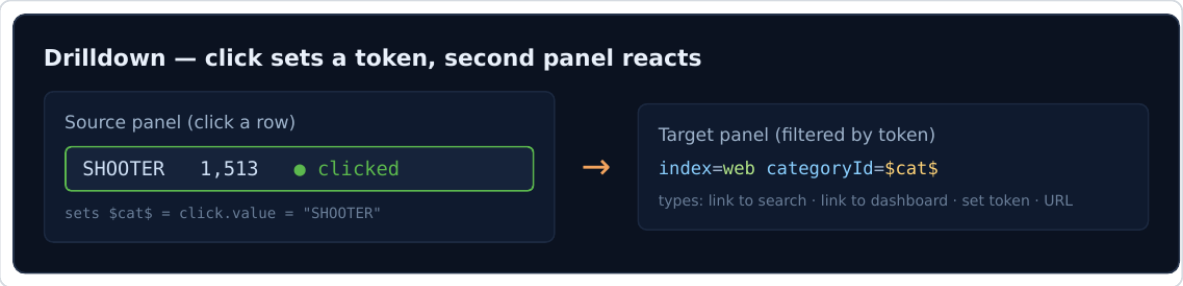


Fig 13.1 — Clicking a category sets cat; the target panel's search filters on it instantly.

Types of drilldown

Type	Behaviour
Set tokens (in-page)	Update tokens so other panels re-run — the interactive "click to filter" pattern.
Link to search	Open the underlying search in a new tab for ad-hoc exploration.
Link to dashboard	Navigate to another Studio/Classic dashboard, passing tokens as URL params.
Link to custom URL	Jump to an external system (ticketing, asset DB), interpolating clicked values.
None	Disable drilldown on decorative panels.

Using tokens in a target search

```
# target panel reacts to the token set by the source panel's click
index=web categoryId=$cat$
| timechart span=1d sum(price) AS revenue
```

Contextual drilldown — carry meaning, not just a value

A **contextual** drilldown passes the *right* context: the clicked category *and* the dashboard's current time range and other active tokens, so the destination opens already scoped. In a URL drilldown, interpolate multiple tokens:

```
# open a detail dashboard scoped to the click AND the current filters
/app/buttercup/category_detail?
form.cat=$click.value$&form.time.earliest=$earliest$&form.time.latest=$latest$
```

TOKEN DEFAULTS PREVENT BROKEN LINKS

Always define what a token means before any click (a sensible default or an `unset` handler). A drilldown that fires with an empty `$cat$` should degrade gracefully, not query nonsense.

ESCAPING IN URLS

Use `$click.value|u$` (URL-encode) when interpolating clicked values into a custom URL, so a value with spaces or symbols doesn't break the link or open an injection hole.

PART E

Dynamic Visualizations

Colour should carry meaning. Static colour paints a series the same regardless of value; **dynamic colour** maps the value itself to a colour — red when a KPI breaches, green when it's healthy — so a dashboard communicates state at a glance.

14 · Dynamic Coloring & Syntax

In Dashboard Studio, dynamic colour is configured as **colour rules** in the visualization's JSON options: you map ranges (or exact matches) of a value to colours. The visualization then colours each cell, bar, or single value by the data it shows — no manual recolouring as numbers change.

Static vs dynamic

Approach	Behaviour
Static colour	Fixed palette per series; identity only ("SHOOTER is green"). Value-independent.
Dynamic colour	Colour chosen from the value via thresholds/matches; encodes state (">5% errors = red").

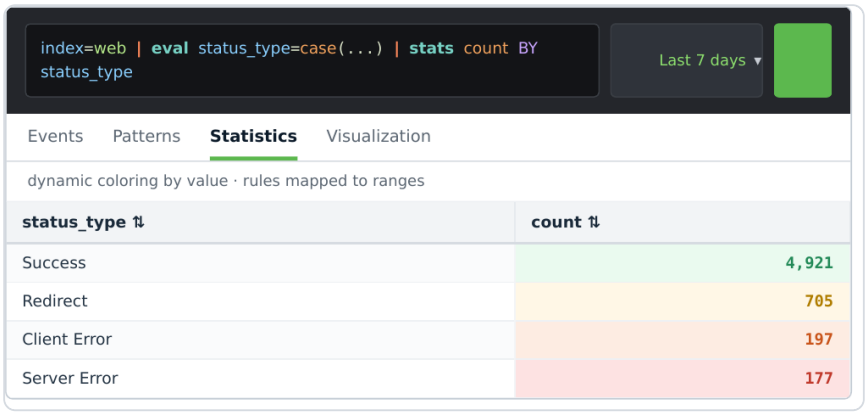


Fig 14.1 — Status families coloured by severity; the eye lands on the red server-error row instantly.

Dynamic colour by threshold (Studio JSON)

```
# colour a Single Value / column by numeric thresholds
{
  "type": "minMidMax",        # or "thresholds" / "categorical"
  "field": "error_rate",
  "thresholds": [
    { "value": 1, "color": "#2fa36b" },   # healthy
    { "value": 3, "color": "#f2a35e" },   # warn
    { "value": 5, "color": "#c0392b" }    # breach
  ]
}
```

Or drive colour from the search

For full control you can emit a colour/severity field in SPL and map the categorical values to colours in the viz options:

```
index=web
| eval status_type = case(status<400,"ok", status<500,"client_err", true(),"server_err")
| stats count BY status_type
```

COLOUR WITH INTENT

Reserve red for "act now". If everything is colourful, nothing stands out. A calm palette with one alert colour reads faster than a rainbow — and stays accessible for colour-blind viewers when paired with labels or icons.

15 · Appendix A — Reporting Command Decision Flowchart

What should shape your results? Start from the x-axis and the number of dimensions.

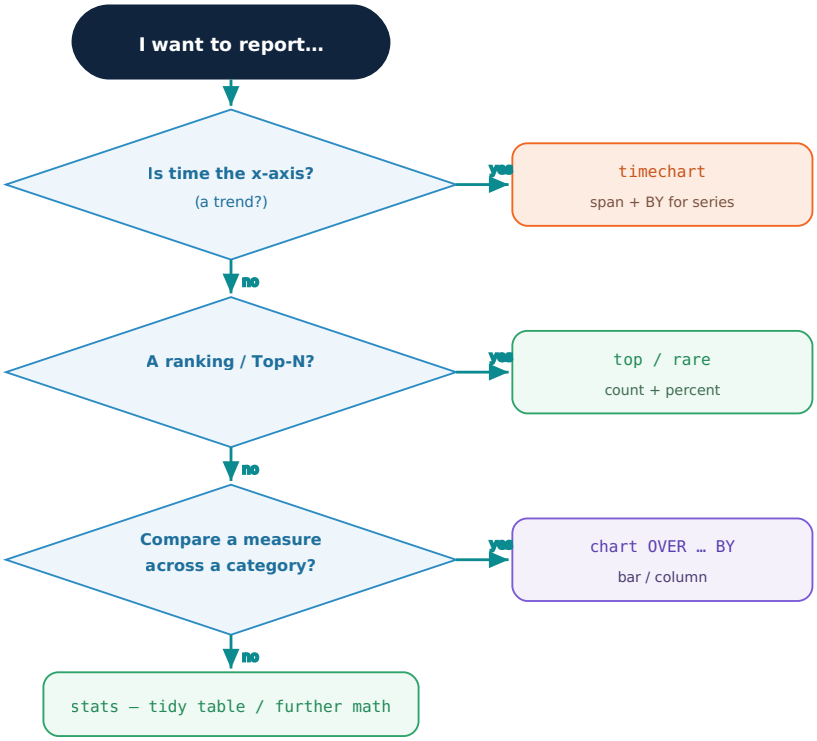


Fig A.1 — Pick the reporting command from the shape of the answer.

16 · Appendix B — Day-4 Quick-Reference Cheat Sheet

REPORTING

```
| chart count BY categoryId
| chart count OVER categoryId BY action
| chart ... limit=5 useother=f usenull=f
| timechart span=1d sum(price) BY categoryId
| top categoryId limit=3
```

EVAL — CALC & FORMAT

```
| eval total=round(price*1.18,2)
| eval fmt="$".tostring(total,"commas")
| eval mb=round(bytes/1024/1024,2)
| eval tier=if(total>=80,"large","std")
| where tier="large"
```

DASHBOARD STUDIO — TOKENS

```
# input sets $cat$; panel search uses it
index=web categoryId=$cat$ | timechart count
# cascading child populate search
index=web categoryId=$cat$ | stats count BY
product_name
# drilldown click tokens
$click.value$  $row.categoryId$  $click.name$
# url drilldown (encode!)
/app/.../detail?form.cat=$click.value|u$
```

DYNAMIC COLOUR (STUDIO JSON)

```
"thresholds": [
  {"value":1,"color":"#2fa36b"},
  {"value":5,"color":"#c0392b"}
]
```

PICK A COMMAND

X-axis / goal	Use
Time	timechart
Category	chart
Ranking	top/rare
Just numbers	stats

END OF DAY 4

You can shape results with chart and timechart, split and clean series, calculate and format with eval, and assemble interactive Dashboard Studio apps with dynamic/cascading inputs, token-driven drilldowns and value-based colour — all on the Buttercup Games dataset.